Docker for Cloud Computing

Comprehensive lecture notes with practical examples

Prepared from the supplied university lecture slides

25 October 2026

Contents

1 Virtualization: VMs and containers **3**
- 1.1 Virtualization and virtual machines . . . 3
- 1.2 Containers . . . 3
- 1.3 Linux primitives: namespaces and cgroups . . . 3

2 Docker concepts and architecture **4**
- 2.1 Docker, images, containers, registries . . . 4
- 2.2 Layered filesystem and copy-on-write . . . 4
- 2.3 Docker Engine client/server architecture . . . 4
- 2.4 Basic command help . . . 4

3 Working with images **4**
- 3.1 Searching, downloading, listing, inspecting . . . 4
- 3.2 Tagging, logging in, and publishing . . . 5

4 Running and managing containers **5**
- 4.1 The meaning of `docker run` . . . 5
- 4.2 Container listings and statistics . . . 6
- 4.3 Executing commands in a running container . . . 6
- 4.4 Container lifecycle commands . . . 6
- 4.5 Container logs and inspection . . . 7

5 Resource constraints and system administration **7**
- 5.1 Docker system commands . . . 7
- 5.2 Memory and CPU limits . . . 7

6 Persistent data: volumes and mounts **8**
- 6.1 Why container writable layers are not enough . . . 8
- 6.2 Three mount types . . . 8
- 6.3 Mount syntax . . . 8
- 6.4 Volume management commands . . . 9
- 6.5 Practical storage examples . . . 9

7 Building images and Dockerfiles **9**
- 7.1 Build context and image creation . . . 9
- 7.2 The `.dockerignore` file . . . 10
- 7.3 Dockerfile instructions . . . 10
- 7.4 FROM, LABEL, COPY, ADD, RUN . . . 11
- 7.5 ARG, ENV, and variable replacement . . . 11
- 7.6 WORKDIR, USER, VOLUME, EXPOSE . . . 11

7.7 ENTRYPOINT and CMD . . . 12
7.8 ONBUILD and multi-stage builds . . . 12
7.9 A complete illustrative Dockerfile . . . 12
7.10 Build from a URL and build promotion . . . 13

8 Docker networking **13**
8.1 Container Network Model . . . 13
8.2 Network commands . . . 13
8.3 Default bridge and user-defined bridge . . . 13
8.4 Aliases, discovery, and legacy links . . . 14
8.5 Built-in network drivers . . . 14
8.6 Port publishing . . . 15

9 Docker Compose and multi-service applications **15**
9.1 Services versus containers . . . 15
9.2 Compose file structure . . . 15
9.3 Core Compose commands . . . 16
9.4 A database and REST API example . . . 16
9.5 Compose lifecycle and scaling . . . 17

10 Databases, application containers, and operational patterns **18**
10.1 Environment variables and a MySQL-compatible database . . . 18
10.2 Using containers as development tools . . . 18

11 Command reference: consolidated list **19**

12 Practical cautions and study checklist **20**

1 Virtualization: VMs and containers

1.1 Virtualization and virtual machines

Virtualization emulates hardware or software components so that a program, or even a complete operating system, behaves as if it had a particular physical machine. A **virtual machine (VM)** presents an emulated hardware platform on which a full guest operating system runs. A **hypervisor** (virtual machine monitor) creates and manages VMs, dividing physical CPU, memory, storage, and networking among them. Examples from the slides include VirtualBox, KVM, VMware, and Xen.

VMs offer strong isolation, but the abstraction has overhead. Sensitive operations can trap to the hypervisor for validation or emulation; memory translation and device I/O also add work. Modern hardware reduces this cost, but each VM still carries a complete guest OS. In a multi-tier deployment, separate VMs might host a gateway, frontend, application services/workers, database, cache, and message broker. This gives useful isolation but can waste resources and makes scaling coarse-grained: scaling one small service may mean replicating an entire VM.

1.2 Containers

A **container** packages an application with its libraries, tools, and configuration in an isolated execution environment. Unlike a VM, a conventional Linux container does not boot a separate kernel: containers share the host kernel. They are therefore generally lighter and faster to start, making them a natural fit for microservices and denser deployments. A container is still a process (or process tree) in the host OS, subject to isolation and resource controls; it is not a complete machine.

	Virtual machine	Container
Kernel	Guest OS and its own kernel	Shares host kernel (for Linux containers)
Packaging	Virtual hardware plus complete OS disk	Application filesystem layers and configuration
Startup/resource footprint	Usually heavier	Usually lighter
Isolation boundary	Virtual hardware/hypervisor	OS-kernel mechanisms such as namespaces and cgroups
Typical unit of scaling	VM	Application/container instance

1.3 Linux primitives: namespaces and cgroups

Namespaces give processes different views of system resources. The slides list:

- PID namespace: process identifiers and process visibility.
- NET namespace: network interfaces, routes, and network stack.
- MNT namespace: mount points and filesystem view.
- UTS namespace: hostname and domain-name view.
- IPC namespace: inter-process communication objects such as semaphores.
- USER namespace: user and group IDs.

Control groups (cgroups) account for and constrain resource use, including memory, CPU, block I/O, and (depending on configuration) network-related accounting. Isolation and resource limits are distinct: a container may be isolated but still have no explicit CPU or memory cap.

2 Docker concepts and architecture

2.1 Docker, images, containers, registries

Docker automates packaging and running applications in containers. Its goal is a uniform way to build, ship, and run applications across environments.

- **Image**: read-only template containing application code, libraries, dependencies, filesystem layers, and metadata. Images can be based on other images and can share layers.
- **Container**: a created/running instance of an image. A container adds a thin writable layer above the image layers; it is intended to be replaceable and often ephemeral.
- **Registry**: service for storing and distributing images. Registries can be public or private. Docker Hub is the default registry used by the standard CLI when no other registry is specified.
- **Runtime/engine**: software that creates and manages containers, networks, images, and volumes, and exposes an API and CLI.

2.2 Layered filesystem and copy-on-write

Image filesystems are assembled from immutable, read-only layers. Layers can be shared across images, and many containers can be created from one image without copying its whole filesystem. When a container changes a file, the changed data is written to that container's writable layer (copy-on-write); unchanged files remain in lower layers. This reduces duplication and allows fast starts. Removing a container removes its writable layer, so changes stored only there disappear.

2.3 Docker Engine client/server architecture

The Docker Engine has three major pieces:

- `dockerd`: the long-running daemon that manages Docker objects.
- REST API: interface through which clients request operations. Communication may use a Unix socket or network connection.
- `docker`: command-line client, which calls the API.

The client and daemon may run on the same machine or on different machines. Beneath the daemon, **containerd** coordinates container lifecycle tasks such as image handling, filesystem preparation, and supervision. **runc**, an OCI-compatible low-level runtime, creates and starts the container process using kernel features such as namespaces and cgroups. The OCI (Open Container Initiative) defines standards that support interoperability among container tooling.

2.4 Basic command help

```
docker --version        # Show installed Docker CLI version
docker -v              # Short version of the same command
docker --help          # List top-level commands and options
docker run --help      # Show options for a particular command
```

The optional word `container` or `image` makes a command's object explicit in many command families: for example, `docker container ls` and `docker ps` list containers; `docker image ls` and `docker images` list images. The slides use both long and short forms.

3 Working with images

3.1 Searching, downloading, listing, inspecting

```
docker search ubuntu
# Filter search results (supported filters depend on CLI/version):
docker search --filter is-official=true ubuntu
```

```
docker search --filter stars=100 ubuntu
# Limit displayed search results:
docker search --limit 5 ubuntu

docker pull alpine:3.9.4
# Select a platform explicitly when needed:
docker pull --platform=linux/amd64 alpine:3.9.4

docker image ls                 # List local images
docker images                   # Legacy shorthand
docker image ls -a              # Include intermediate/dangling images
docker image ls -q              # Print image IDs only
docker image ls --no-trunc      # Show full IDs

docker image inspect alpine:3.9.4
docker image inspect --platform=linux/amd64 alpine:3.9.4 \\
  --format '{{json .RootFS.Layers}}'
```

`docker search` searches registry listings. Filters can select official images or minimum star counts, and `-limit` controls the number of results. `docker pull` downloads an image and any missing layers. If two images share a layer, Docker can reuse the local copy rather than download it again. A tag such as `alpine:3.9.4` identifies a version; if no tag is supplied, Docker generally uses `latest`, which is a mutable tag rather than a guarantee of newest or safest software. `inspect` returns structured metadata; `-format` extracts a specific field using a Go template.

3.2 Tagging, logging in, and publishing

Images are named with a repository and optional tag. A registry-qualified name can include a registry hostname and namespace, for example `registry.example.com/team/app:1.2`. Common publication flow:

```
docker login                          # Authenticate to a registry
docker tag local-app:1.0 USER/local-app:1.0
docker push USER/local-app:1.0        # Upload image and required layers
```

The source slides focus on build and push workflows. Authenticate only to a registry you trust, and do not put passwords directly in shell history or Dockerfiles.

4 Running and managing containers

4.1 The meaning of `docker run`

`docker run` combines several actions: if needed it pulls the image, creates a container, configures it, and starts its main process. The container's lifetime is tied to that process: when the main process exits, the container stops.

```
docker run hello-world                 # Pull if needed, run, then exit
docker run --name web -d nginx         # Named container, detached
docker run --name web -p 8081:80 -d nginx:latest
```

In `-p 8081:80`, port 8081 on the host is forwarded to port 80 in the container. The application inside the container must listen on the container port. `-d` runs in the background. `-name` assigns a human-readable name. `-it` combines an interactive stdin connection with a pseudo-terminal; it is useful for shells. `-rm` automatically removes the container after it exits. These options can be combined, e.g. `docker run -rm -it alpine sh`.

4.2 Container listings and statistics

```
docker ps                          # Running containers
docker ps -a                       # All containers, including stopped ones
docker container ls                 # Equivalent to docker ps
docker container ls -a              # Include stopped containers
docker ps -s                       # Show writable/root filesystem size
docker ps -as                      # Show sizes for all containers
docker stats                       # Live resource-use stream
docker stats CONTAINER_ID           # One container
docker top CONTAINER_NAME           # Processes running in a container
```

The `ps` output typically includes container ID, image, command, creation time, status, published ports, and name. `-s` adds filesystem-size information; the writable size is not the same as total image size. `stats` reports live CPU, memory, network I/O, and block I/O metrics. `top` displays processes in the container from the host's perspective.

4.3 Executing commands in a running container

```
docker exec CONTAINER_ID ls
docker exec -d CONTAINER_ID some-command  # Detached command
docker exec -it CONTAINER_ID bash        # Interactive shell, if bash exists
docker exec -it CONTAINER_ID sh          # More widely available shell
docker exec CONTAINER_ID env             # Print environment variables
```

`exec` starts an additional process inside an already-running container; it does not replace the main process. `-d` detaches, `-i` keeps stdin open, and `-t` allocates a terminal. Minimal images (for example Alpine) may not include Bash, so use `sh` or Alpine's `ash` when needed.

4.4 Container lifecycle commands

Command	Meaning and example
`docker create IMAGE`	Create a container without starting it.
`docker start NAME`	Start a stopped container.
`docker stop NAME`	Ask the main process to stop gracefully (normally SIGTERM, then SIGKILL after a timeout).
`docker restart NAME`	Stop and start the container again.
`docker pause NAME`	Suspend processes in a running container; resources remain allocated.
`docker unpause NAME`	Resume paused processes.
`docker kill NAME`	Send a signal (SIGKILL by default); can specify another signal with `-signal`.
`docker rm NAME`	Remove a stopped container.
`docker rm -f NAME`	Force removal, stopping a running container if necessary.
`docker container remove -f NAME`	Explicit object-family form of forced removal.
`docker run -rm IMAGE`	Remove the container automatically after it exits.

Useful inspection sequence:

```
docker ps
docker ps -a
docker stop web
docker start web
```

```
docker logs web
docker rm web
```

Stopping is not deleting. A stopped container remains listed by `docker ps -a` until removed. Removing a container does not remove the image used to create it.

4.5 Container logs and inspection

```
docker logs CONTAINER
docker logs --follow CONTAINER
docker logs --tail 5 CONTAINER
docker logs --timestamps --details CONTAINER
docker inspect CONTAINER
docker inspect -f '{{ .NetworkSettings.IPAddress }}' CONTAINER
```

`logs` displays the main process's captured standard output/error. `-follow` streams new output; `-tail` limits the number of lines; `-timestamps` adds timestamps; `-details` includes extra attributes when available. `inspect` returns detailed configuration and state, including mounts, networking, environment, and runtime settings. The IP-address template is useful on networks where a container IP is assigned, but applications should usually use service/container names on a user-defined network rather than hard-code dynamic IP addresses.

5 Resource constraints and system administration

5.1 Docker system commands

```
docker system --help
docker system df
docker system df -v
docker system events
docker system prune
docker system prune -a
```

`docker system df` summarizes disk use by images, containers, local volumes, and build cache; `-v` adds detail. `docker system events` streams daemon events such as container start/stop/kill. `docker system prune` removes unused objects to reclaim disk space. Adding `-a` broadens image cleanup to unused images, not just dangling ones. Review the prompt and understand which data is disposable before pruning; volumes can contain persistent data.

5.2 Memory and CPU limits

By default, a container has no explicit CPU or memory limit and may use resources allowed by the host kernel. Docker supports hard limits and soft reservations. A hard memory cap restricts the maximum; a memory reservation is a softer threshold that matters under host contention.

```
# Hard memory limit of 256 MiB
docker run --memory=256m IMAGE
# Soft memory reservation
docker run --memory-reservation=256m IMAGE
# At most half a CPU and 100 MiB RAM
docker run --name halfcpu -d --cpus=0.5 --memory=100m alpine \\
  dd if=/dev/zero of=/dev/null
# Pin to selected logical CPUs
docker run --cpuset-cpus="0-1" IMAGE
# Relative CPU weight when CPU is contended
docker run --cpu-shares=512 IMAGE
```

`-cpus` specifies a quota as a fraction of CPU capacity (for example, 1.5 means at most one and a half CPUs' worth). The lower-level `-cpu-period` and `-cpu-quota` configure the CFS scheduling period and quota; the slides recommend `-cpus` for the common case. `-cpuset-cpus` restricts the CPUs/cores a container may run on (e.g. `0-3` or `1,3`). `-cpu-shares` is a relative weight, typically 1024 by default, and matters when CPU time is contested; it is not a hard cap. `docker stats` can be used to observe the result.

6 Persistent data: volumes and mounts

6.1 Why container writable layers are not enough

Data stored only in a container's writable layer is lost when that container is removed. It is also harder to share with other processes, tied to the container's storage driver, and can be slower than direct host filesystem writes. Persistent application state should generally live in a mount.

6.2 Three mount types

Type	Where data lives	Typical use / caveat
Volume	Docker-managed host storage (on Linux commonly under `/var/lib/docker/volumes/`).	Preferred general-purpose persistent storage; Docker manages lifecycle; may be named or anonymous and may use volume drivers.
Bind mount	An explicit file or directory path on the host.	Share source code/configuration or host files. Depends on host path layout and lets the container potentially modify host files.
tmpfs	Host memory; not written to persistent disk.	Temporary state or sensitive transient data; disappears when the container stops/is removed.

A volume can be mounted into multiple containers. Volume drivers (plugins) extend storage beyond the local host, for example to remote storage or cloud providers; the slides name Azure File Storage, Flocker, GlusterFS, OpenStorage, GCE-Docker, Ceph RBD, and FUSE-derived drivers. Docker can create a volume automatically if a requested named/anonymous volume does not exist. When an empty volume is mounted over a populated image directory, Docker normally copies the existing directory contents into the volume unless copy-up is disabled. Mounting a non-empty volume or bind mount over a directory hides the underlying image files for as long as the mount is present; it does not delete them.

6.3 Mount syntax

```
# Named volume: Docker manages the volume
 docker volume create app-data
 docker run -d --name db --mount type=volume,source=app-data,target=/var/lib/mysql mysql
# Equivalent compact syntax for many common cases:
 docker run -d -v app-data:/var/lib/mysql mysql
# Bind mount the current host directory into /app
 docker run --rm -it --mount type=bind,source="$(pwd)",target=/app alpine sh
# Compact bind mount syntax
 docker run --rm -it -v "$(pwd)":/app alpine sh
# Read-only bind mount
 docker run --mount type=bind,source="$(pwd)/config",target=/app/config,readonly IMAGE
```

```
# tmpfs mount
 docker run --mount type=tmpfs,target=/run/cache IMAGE
```

For `-mount`, comma-separated key/value fields specify `type`, `source`/`src`, `destination`/`dst`/`target`, and optionally `readonly`. The mount type may be omitted for a Docker-managed volume in supported forms. `-v` is shorter but `-mount` makes the mount type and fields explicit.

6.4 Volume management commands

```
docker volume create app-data
docker volume ls
docker inspect app-data
docker volume rm app-data
docker volume prune
```

A volume cannot normally be removed while in use by a container. `docker volume prune` removes unused volumes, so ensure that no needed data is stored in an unattached volume.

6.5 Practical storage examples

Persist a database directory independently of a container:

```
docker volume create mysql-data
docker run -d --name mysql-db \\
  -e MYSQL_ROOT_PASSWORD='change-me' \\
  -v mysql-data:/var/lib/mysql mysql:8
docker inspect mysql-db
docker rm -f mysql-db
# Re-create using the same mysql-data volume to reuse the database files.
```

Share a local project directory with a tool container:

```
docker run --rm -it -v "$(pwd)/project":/app -w /app \\
  maven:3.5-jdk-8 mvn clean install -Dmaven.test.skip=true
```

Here `-w /app` sets the working directory. The container is removed after the command finishes, while files written through the bind mount remain on the host. For a Python script requiring NumPy, the same pattern is `docker run -rm -it -v "$(pwd)/es-python":/app -w /app adreeve/python-numpy python es-python.py`.

7 Building images and Dockerfiles

7.1 Build context and image creation

`docker build` reads a Dockerfile and a **build context**, a set of files at a local path or URL that the builder can access. The instructions are processed in order to create an image with filesystem layers and metadata such as default process, exposed ports, environment variables, user, and volume mount points.

```
docker build -t myapp:1.0 .
docker build -t myapp:1.0 -f path/to/Dockerfile .
docker build --pull --no-cache -t myapp:dev .
docker build --quiet -t myapp:1.0 .
docker build --target build -t myapp-build .
```

The final `.` is the context directory. `-t`/`-tag` assigns a name and tag; `-f`/`-file` selects a Dockerfile; `-pull` attempts to refresh the base image; `-no-cache` (additional common option) disables reuse of cached build steps; `-quiet` prints less build output; `-rm` removes intermediate containers after a

successful build; `-force-rm` removes them even if the build fails; `-build-arg NAME=value` supplies a build argument; `-target` selects a named stage in a multi-stage build. Multiple tags can point to the same image ID.

Build-time resource flags in the lecture include `-cpu-period`, `-cpu-quota`, `-cpu-shares/-c`, `-cpuset-cpus`, `-memory/-m`, and `-memory-swap`. They constrain build instruction execution, not the resources of the final container. BuildKit/buildx behavior can vary with Docker version; consult the installed CLI help for exact current options.

7.2 The `.dockerignore` file

A `.dockerignore` file is not a Dockerfile instruction. It is a newline-separated list of patterns that excludes matching files from the build context, reducing transfer time and preventing unnecessary files or secrets from being sent to the builder. Example:

```
# Ignore temporary directories and Markdown files
*/temp*
*/*/temp*
temp?
*.md
!README*.md
README-secret.md
```

Patterns are applied according to Docker's ignore rules; negation patterns beginning with `!` can re-include selected paths. Keep credentials and local build artifacts out of the context.

7.3 Dockerfile instructions

Each instruction generally has the form `INSTRUCTION arguments`; lines beginning with `##` are comments. Docker processes instructions in sequence. Some affect build-time layers, others configure container runtime behavior.

Instruction	Purpose
`FROM`	Select base image and begin a build stage. `FROM scratch` starts from an explicitly empty base.
`LABEL`	Add key/value metadata; labels from a base may be inherited and a later value can override an earlier one.
`COPY`	Copy files/directories from the build context (or a named stage) into the image.
`ADD`	Copy files and has additional behavior such as supported remote URL/local archive handling. Prefer `COPY` unless the extra behavior is needed.
`RUN`	Execute a build-time command in a temporary build container; its filesystem changes form a layer.
`ONBUILD`	Register an instruction to run when a downstream image uses this image as its base.
`ARG`	Define a build-time variable, optionally with a default; pass values using `-build-arg`. Not automatically persisted as a runtime environment variable.
`ENV`	Set environment variables available to later build instructions and containers created from the image.
`WORKDIR`	Set the working directory for following `RUN`, `CMD`, `ENTRYPOINT`, `COPY`, and `ADD`; creates it if needed. Prefer absolute paths.
`VOLUME`	Declare a mount point intended for external storage. Docker may initialize an anonymous volume from image contents.

Instruction	Purpose
USER	Set the user and optionally group used by subsequent runtime commands and relevant later build instructions.
EXPOSE	Document a port the application listens on; does not publish it to the host.
ENTRYPOINT	Define the main executable/process for the container. Exec form is generally preferred.
CMD	Provide default command or default arguments to ENTRYPOINT; only the last CMD in a Dockerfile takes effect.
HEALTHCHECK	Define a command Docker uses to assess container health (supported by Dockerfile syntax).
STOPSIGNAL	Set the signal sent when stopping the container.
SHELL	Change the shell used for shell-form instructions on supported platforms.

MAINTAINER appears in older material but is deprecated in favor of LABEL metadata.

7.4 FROM, LABEL, COPY, ADD, RUN

```
FROM ubuntu:24.04
LABEL org.opencontainers.image.authors="Cloud Computing laboratory"
WORKDIR /opt/app
COPY requirements.txt .
RUN apt-get update \\
 && apt-get install -y --no-install-recommends curl ca-certificates \\
 && rm -rf /var/lib/apt/lists/*
COPY . .
```

FROM establishes the base filesystem. COPY source destination copies from the context into the image. ADD can download a URL or unpack a local tar archive, so its extra semantics should be deliberate. RUN executes at build time; chaining related package-manager operations with && can reduce unnecessary layers and allows cleanup in the same layer. Build context paths and destination paths must be considered carefully.

7.5 ARG, ENV, and variable replacement

```
ARG APP_VERSION=1.0
FROM alpine:3.20
ARG APP_VERSION
ENV APP_HOME=/opt/app APP_VERSION=${APP_VERSION}
WORKDIR ${APP_HOME}
```

An ARG is supplied during build, for example docker build -build-arg APP_VERSION=2.0 -t app:2.0 .. An ENV variable persists in image configuration and is available when a container runs. Do not use either as a safe place for secrets: build arguments and image environment metadata may be exposed. Dockerfile variable forms include $NAME and ${NAME}. The forms ${NAME:-word} and ${NAME:+word} provide shell-like default/conditional substitution in supported Dockerfile variable-replacement contexts: the first uses word if the variable is unset/empty according to the relevant semantics; the second expands to word when set and otherwise to an empty string.

7.6 WORKDIR, USER, VOLUME, EXPOSE

```
WORKDIR /srv/app
COPY app.jar .
USER 10001:10001
```

```
VOLUME ["/data"]
EXPOSE 8080
```

Use `WORKDIR` rather than relying on a shell command such as `RUN cd ...`; each instruction runs in its own build step. `USER` improves security by avoiding root where possible. `VOLUME` marks a path for external data storage. `EXPOSE 8080` documents the container port, but access from the host still requires publishing, for example `docker run -p 8080:8080 IMAGE`; `-P` publishes all exposed ports to randomly selected host ports.

7.7 ENTRYPOINT and CMD

`ENTRYPOINT` defines the executable that runs when a container starts. `CMD` supplies default arguments to that executable, or a default command when no entrypoint is set. Prefer JSON/exec form because it launches the executable directly and handles signals more predictably:

```
ENTRYPOINT ["java", "-jar", "app.jar"]
CMD ["--server.port=8080"]
```

The shell form runs through a shell (such as `/bin/sh -c`), which can interfere with signal delivery and graceful shutdown. Only the last `CMD` is effective. Runtime arguments normally replace `CMD`'s defaults and are passed to an exec-form entrypoint. Override an entrypoint with `docker run -entrypoint=/bin/sh IMAGE`; use carefully because this replaces the intended application process.

7.8 ONBUILD and multi-stage builds

`ONBUILD` defers an instruction until another Dockerfile builds `FROM` the current image. It is useful for reusable base images that automate steps for downstream projects.

A multi-stage build uses multiple `FROM` instructions. Each starts a new stage; selected artifacts are copied into the final stage, leaving compilers, build caches, and development dependencies behind.

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /src
COPY pom.xml .
COPY src ./src
RUN mvn -B package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /src/target/app.jar ./app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

`docker build -target build -t app-build .` builds only the named intermediate stage. Multi-stage builds make runtime images smaller and reduce the software included in the final artifact.

7.9 A complete illustrative Dockerfile

```
FROM nginx:1.18.0
LABEL org.opencontainers.image.title="Static web demo"
COPY html/ /usr/share/nginx/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Build and run it:

```
docker build -t my-nginx:1.0 .
docker run --name my-site -d -p 8080:80 --memory=500m my-nginx:1.0
curl http://localhost:8080
docker logs my-site
```

```
docker rm -f my-site
```

The example keeps NGINX in the foreground so it remains the container's main process. The course also demonstrates a MongoDB-oriented image with package installation in `RUN`, a non-root `USER mongodb`, `VOLUME ["/data/db"]`, `EXPOSE 27017`, `ENTRYPOINT ["/usr/bin/mongod"]`, and `CMD ["-port", "27017", "-dbpath", "/data/db", "-bind_ip_all"]`. That example illustrates how build steps, data paths, ports, user, and runtime defaults fit together.

7.10 Build from a URL and build promotion

Docker can use a Git repository URL as a build context. The repository is fetched into a temporary local directory and sent to the builder. Example from the slides:

```
docker build -t USER/nginx-example github.com/atbaker/nginx-example
```

After building and testing, tag and push an image to a registry to promote it between environments. Reusing the same immutable image artifact across test/staging/production improves consistency; configuration should be injected at runtime rather than baked in when it varies by environment.

8 Docker networking

8.1 Container Network Model

Docker's Container Network Model (CNM) expresses networking through four components:

- **Sandbox**: a container's network stack configuration, including interfaces, IP addresses, routes, and DNS settings. It is commonly implemented with a Linux network namespace.
- **Endpoint**: connects one sandbox to one network. A Linux virtual Ethernet (veth) pair is a common implementation; one end resides in the container namespace and the other attaches to the network.
- **Network**: a group of endpoints that can communicate, implemented for example by a Linux bridge or overlay.
- **Cluster**: Docker hosts connected by one or more networks.

A Linux network namespace has its own interfaces, routes, and firewall rules. A container may join multiple networks. **libnetwork** implements CNM, supplies pluggable network drivers and IP address management (IPAM), and supports service discovery and load-balancing features. CNM is distinct from the Container Network Interface (CNI) used by systems such as Kubernetes.

8.2 Network commands

```
docker network ls
docker network create -d bridge my-net
docker network create --driver bridge --subnet 172.25.0.0/16 my-net
docker network inspect my-net
docker network connect my-net CONTAINER
docker network disconnect my-net CONTAINER
docker network rm my-net
```

`create` creates a network; `-d bridge` selects the bridge driver; `-subnet` specifies an IP address range. `inspect` shows configuration and connected endpoints. `connect` and `disconnect` attach/detach a container to/from an existing network. A network generally cannot be removed while containers remain connected.

8.3 Default bridge and user-defined bridge

On Linux, Docker's default bridge uses a virtual bridge commonly named `docker0`; each attached container typically has a veth pair and an interface such as `eth0`. IP forwarding and firewall/NAT

rules provide external connectivity. The default bridge is convenient for simple experiments, but name-based service discovery is limited compared with user-defined networks. Docker recommends user-defined bridge networks for applications because containers can resolve one another by name or network alias and can be attached/detached dynamically.

```
docker network create --driver bridge --subnet 172.25.0.0/16 my-net
docker run -itd --name=container1 busybox
docker run -itd --name=container2 busybox
docker network connect my-net container2
docker run --network=my-net --ip=172.25.3.3 -itd --name=container3 busybox
docker exec -it container2 sh
# Inside the container:
ping container3
```

The example uses a static IP inside the declared subnet. Static assignments must be unique and belong to the network's subnet. Prefer DNS names/aliases for most applications because container IP addresses can change.

8.4 Aliases, discovery, and legacy links

User-defined networks provide embedded DNS. A container can connect to a service by its container name or configured network alias. Multiple containers can share an alias; this can support simple distribution across matching endpoints, though a production load-balancing design needs deliberate health and routing behavior.

```
docker network create my-net
docker run -itd --network=my-net --name c1 --network-alias foo busybox
docker run -itd --network=my-net --name c2 --network-alias bar busybox
docker run -itd --network=my-net --name c3 --network-alias foo busybox
docker exec c2 ping foo
```

The older `-link=CONTAINER:ALIAS` option added name resolution, aliasing, and environment-variable injection on the default bridge. It is a legacy feature; user-defined networks are preferred.

8.5 Built-in network drivers

Driver/mode	Behavior and trade-offs
`none`	No external networking; loopback remains. Useful where networking is deliberately disabled.
`bridge`	Default single-host network based on a Linux bridge/virtual switch. Good for development and same-host container communication.
`host`	Container shares the host network namespace and IP stack. Port isolation is removed; processes must coordinate host ports, and exposure/security require care.
`overlay`	Connects containers across multiple Docker hosts using an overlay network (for example VXLAN encapsulation). Requires an appropriate multi-host setup.
`macvlan`	Gives a container its own MAC address so it appears like a physical device on the LAN; useful for some legacy applications but depends on network infrastructure.

Examples:

```
docker run --network=none -it --name isolated busybox sh
docker run --network=host IMAGE
docker network create --driver bridge my-net
```

Host networking is not equivalent to normal port publishing: the container uses the host's ports directly, so two processes cannot bind the same address/port combination. Overlay networks support cross-host communication; the driver handles routing to the correct Docker host and destination container.

8.6 Port publishing

```
docker run -d --name nginx -p 8081:80 nginx
docker run -d --name nginx-random -P nginx
```

`-p HOST_PORT:CONTAINER_PORT` explicitly publishes a port. `-P` publishes all ports declared with `EXPOSE` to randomly selected host ports. `EXPOSE` alone does not make a port reachable from the host. Binding to `127.0.0.1:8080:8080` limits access to the host loopback interface, which is often safer for local development than publishing on every host interface.

9 Docker Compose and multi-service applications

9.1 Services versus containers

A distributed application is made of services, such as a frontend/API and a database. A Compose service is a declarative definition of how to run a component; it may have one or more container replicas. Scaling a service horizontally changes the number of instances. Compose makes multi-container development environments repeatable, isolated, and quick to start, and can be used in CI/CD workflows alongside tools such as Maven, JUnit, or Jenkins.

9.2 Compose file structure

Compose configuration is YAML, commonly named `compose.yaml` or `docker-compose.yml`. Environment configuration can be supplied through an environment file (for example with a service-level `env_file:`) or variable interpolation from a project `.env` file. The course compares development and production files such as `.env.dev` and `.env`; remember that Compose interpolation and variables passed into the container are related but distinct mechanisms, and verify precedence for the Compose version in use. The slides use the older Compose file format with a `version` key; current Compose implementations generally use the Compose Specification and may treat that key as obsolete. The key concepts remain `services`, `networks`, and `volumes`.

```
services:
  web:
    build:
      context: .
      dockerfile: Dockerfile
    image: demochatweb:0.1
    ports:
      - "8080:9000"
    networks:
      - my-net
  mongo:
    image: mongo:latest
    volumes:
      - data:/data/db
    networks:
      - my-net
networks:
  my-net:
    driver: bridge
volumes:
```

```
data:
  driver: local
```

`services` defines application components and their image/build settings, ports, environment, dependencies, mounts, and networks. `build.context` identifies the build context and `build.dockerfile` chooses the Dockerfile. `image` names the resulting or pulled image. `ports` publishes host-to-container ports. `volumes` mounts persistent data. `networks` connects services to named networks. Top-level `volumes` and `networks` declare resources shared by services. Compose creates a project network by default even when a custom network is not explicitly declared.

9.3 Core Compose commands

Command	Purpose
`docker compose up`	Create/start the application's services; `-d` detaches.
`docker compose down`	Stop and remove project containers and networks; named volumes are normally retained unless volume removal is requested.
`docker compose build`	Build or rebuild service images.
`docker compose pull`	Pull service images.
`docker compose config`	Validate and render the resolved configuration.
`docker compose create`	Create service containers without starting them.
`docker compose start / stop`	Start/stop existing service containers.
`docker compose restart`	Restart services.
`docker compose ps`	List project containers and their state.
`docker compose logs`	Display logs; `-f` follows output.
`docker compose exec SERVICE CMD`	Execute a command in a running service container.
`docker compose run SERVICE CMD`	Start a one-off container for a service command.
`docker compose top`	Show processes for service containers.
`docker compose images`	List images used by the project.
`docker compose port SERVICE PORT`	Show the published host port for a service port.
`docker compose pause / unpause`	Pause/resume service containers.
`docker compose kill`	Send a kill signal to service containers.
`docker compose rm`	Remove stopped service containers.
`docker compose events`	Stream container events.
`docker compose push`	Push service images to their registries.
`docker compose scale SERVICE=N`	Set a service's replica count (legacy form; `up -scale` is common).
`docker compose version`	Display Compose version information.

Use `-f path/to/compose.yaml` to select a different file. If the default file exists in the current directory, it need not be specified.

9.4 A database and REST API example

The lecture illustrates a REST service backed by MariaDB. The database should be on a private user-defined network; only the API needs a published host port. Use persistent storage for the database and environment variables for configuration.

```
services:
  db:
    image: mariadb:11.4
    environment:
      MARIADB_ROOT_PASSWORD: ${MARIADB_ROOT_PASSWORD}
      MARIADB_DATABASE: db_micro
      MARIADB_USER: micro
      MARIADB_PASSWORD: ${MARIADB_PASSWORD}
    volumes:
      - mariadb-data:/var/lib/mysql
    networks: [backend]
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      interval: 10s
      timeout: 5s
      retries: 10
  api:
    image: miciav/micro:0.3.0
    environment:
      MYSQL_ADDRESS: db
      MYSQL_PORT: "3306"
      DB_USERNAME: micro
      DB_PASSWORD: ${MARIADB_PASSWORD}
    ports:
      - "127.0.0.1:8080:8080"
    networks: [backend]
    depends_on:
      db:
        condition: service_healthy
networks:
  backend:
    driver: bridge
volumes:
  mariadb-data:
```

This is an illustrative, adapted configuration; image-specific health-check commands and environment-variable names must be verified for the selected image. A plain `depends_on` can establish startup order but does not by itself guarantee that a database is ready to accept connections; a health check and readiness condition address that distinction. Store sensitive values in an appropriate secret manager or local environment file excluded from version control, rather than committing passwords.

9.5 Compose lifecycle and scaling

```
docker compose up -d
docker compose ps
docker compose logs -f
docker compose exec api sh
docker compose up -d --scale api=2
docker compose down
```

A service with a fixed host-port binding generally cannot be scaled to multiple replicas using that same host port, because replicas would compete for it. Scaling is most useful for services behind a reverse proxy/load balancer or services that do not bind the same fixed host port. The slides demonstrate an NGINX proxy routing by HTTP Host header to replicas, and use `curl -H "Host: whoami.local" localhost` to test host-based routing.

10 Databases, application containers, and operational patterns

10.1 Environment variables and a MySQL-compatible database

Container environment variables are supplied at runtime with `-e KEY=value` or `-env-file FILE`; inside a running container they can be inspected with `docker exec CONTAINER env`. The slides show MySQL-compatible configuration variables such as `MYSQL_ROOT_PASSWORD`, `MYSQL_ROOT_HOST`, `MARIADB_ROOT_PASSWORD`, `MARIADB_DATABASE`, `MARIADB_USER`, and `MARIADB_PASSWORD`. The exact variables depend on the image.

An example launch pattern (replace credentials and check the chosen image documentation):

```
docker run -d --name mysql-server -p 3306:3306 \\
  -e MYSQL_ROOT_PASSWORD='change-me' \\
  -e MYSQL_ROOT_HOST='%' mysql/mysql-server:8.0.20
```

Then open a shell and database client, if installed:

```
docker exec -it mysql-server bash
mysql -uroot -p
```

The course SQL examples create a database and grant access:

```
CREATE DATABASE cloud;
CREATE USER 'user'@'%' IDENTIFIED BY 'replace-this-password';
GRANT ALL PRIVILEGES ON cloud.* TO 'user'@'%' WITH GRANT OPTION;
FLUSH PRIVILEGES;
USE cloud;
CREATE TABLE students (
  first_name VARCHAR(30) NOT NULL,
  last_name VARCHAR(60) NOT NULL,
  email VARCHAR(256) NULL,
  address VARCHAR(256) NOT NULL,
  student_id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY
);
INSERT INTO students (first_name, last_name, address) VALUES ('Ada', 'Lovelace', 'London')
    ;
SELECT * FROM students;
```

SQL data held only in the container's writable layer disappears when the container is removed. Mount a named volume at the database's data directory to preserve it. Avoid exposing database ports publicly unless required; on a private Docker network, application containers can reach the database by service name without publishing its port on the host.

10.2 Using containers as development tools

A container can be a disposable compiler/runtime environment. For Maven compilation with a host project mounted into `/workspace`:

```
docker run --rm -v "$(pwd)/es-micro":/workspace -w /workspace \\
  maven:3.5-jdk-8 mvn clean install -Dmaven.test.skip=true
```

Run a generated JAR similarly, publishing its service port:

```
docker run --rm -v "$(pwd)/es-micro/target":/app -w /app \\
  -p 8080:8080 maven:3.5-jdk-8 \\
  java -jar ./micro-0.0.1-SNAPSHOT.jar
```

For a remote lab VM, SSH local port forwarding can make a remote service reachable at local port 8080:

```
ssh -L 8080:localhost:8080 -p VM_PORT student@VM_IP
curl localhost:8080/measure/temperature
curl localhost:8080/measure/temperature | python -m json.tool
```

The shell command `$(pwd)` expands to the absolute path of the current working directory in common POSIX shells. Ensure bind-mount paths are correct for the operating system and shell in use.

11 Command reference: consolidated list

This section groups the commands appearing throughout the lecture material. Placeholder values such as `CONTAINER`, `IMAGE`, `VOL_NAME`, and `NET_NAME` must be replaced with real names or IDs.

Command	Use
`docker -version`, `docker -help`	Version and general help.
`docker search IMAGE`	Search registry listings; supports filters and limit.
`docker pull IMAGE[:TAG]`	Download image/layers.
`docker image ls` / `docker images`	List local images; options `-a`, `-q`, `-no-trunc`.
`docker image inspect IMAGE`	Inspect image metadata and layers.
`docker rmi IMAGE` / `docker image rm IMAGE`	Remove a local image (not a running container).
`docker tag SOURCE TARGET`	Add a repository/tag reference to an image.
`docker login`, `docker push IMAGE`	Authenticate and publish.
`docker build -t NAME:TAG PATH`	Build an image from a Dockerfile/context.
`docker run OPTIONS IMAGE COMMAND`	Create and start a container.
`docker create IMAGE`	Create without starting.
`docker ps [-a] [-s]`	List running/all containers and optionally sizes.
`docker start/stop/restart NAME`	Start, stop, or restart.
`docker pause/unpause NAME`	Suspend/resume container processes.
`docker kill NAME`	Send a signal (SIGKILL by default).
`docker rm [-f] NAME`	Remove container, optionally force.
`docker exec [-it] NAME CMD`	Run command in running container.
`docker top NAME` / `docker stats NAME`	Process list / live resource metrics.
`docker inspect NAME` / `docker logs NAME`	Detailed configuration/state / main-process logs.
`docker system df [-v]`	Disk usage summary/detail.
`docker system events`	Stream daemon events.
`docker system prune [-a]`	Remove unused objects to reclaim space.
`docker volume create/ls/rm/prune`	Manage Docker volumes.
`docker network create/ls/inspect/rm`	Manage networks.
`docker network connect/disconnect NET CONTAINER`	Attach/detach a container.
`docker compose up/down`	Start stack / stop and remove project containers and networks.
`docker compose build/pull/push`	Build, download, or publish service images.
`docker compose ps/logs/exec/run`	Inspect services, view logs, execute or run commands.
`docker compose config`	Validate and render resolved YAML.

12 Practical cautions and study checklist

- Distinguish image from container: an image is the template; a container is an instance with its own writable layer and runtime state.
- Distinguish stopping from removing. `stop` preserves the container; `rm` deletes it. `-rm` removes it after exit.
- Persist state in a volume or bind mount, not only in the container layer.
- Remember that `EXPOSE` documents a port; `-p` publishes it to the host.
- Use user-defined bridge networks for name-based communication between containers on one host.
- Apply resource limits explicitly where workloads require them; `-cpu-shares` is a relative weight, while `-cpus` and `-memory` are limits.
- Prefer exec-form `ENTRYPOINT` and `CMD` for predictable signal handling.
- Keep build contexts small with `.dockerignore`; avoid embedding credentials in image layers or metadata.
- Use multi-stage builds to keep compilers and build dependencies out of runtime images.
- Treat `prune`, volume deletion, and forced container removal as potentially destructive operations.
- In Compose, startup order is not the same as application readiness. Use health checks and robust retry logic for dependencies.